Different Kinds of RAG Architectures and Example Projects

Introduction

Retrieval-Augmented Generation (RAG) is an AI architecture that combines information retrieval systems with Large Language Models (LLMs). Instead of depending only on pre-trained knowledge, RAG systems retrieve relevant external information and use it during response generation.

RAG has evolved into multiple architectures optimized for:

- better retrieval quality,
- reasoning,
- multimodal understanding,
- real-time intelligence,
- enterprise workflows,
- autonomous AI systems.

This document explains the major types of RAG systems, how they work, technologies commonly used, advantages and limitations, and real-world example projects.

---

1. Naive RAG

Overview

Naive RAG is the simplest implementation of Retrieval-Augmented Generation.

The system: 1. Converts documents into embeddings 2. Stores them in a vector database 3. Retrieves relevant chunks for a query 4. Sends them to the LLM

Architecture Flow

```
User Query
   ↓
Embedding Model
   ↓
Vector Search
   ↓
Top-K Chunks
   ↓
LLM
```

```
   ↓
Generated Response
```

Technologies Used

- LangChain
- LlamaIndex
- Pinecone
- ChromaDB
- FAISS
- OpenAI Embeddings
- Sentence Transformers

Advantages

- Easy to build
- Low cost
- Fast implementation
- Good for prototypes

Limitations

- Weak retrieval quality
- Poor reasoning
- Hallucination issues
- No query optimization

Example Projects

Project 1: PDF Question Answering Chatbot

Build a chatbot that:

- uploads PDFs,
- chunks text,
- stores embeddings,
- answers questions.

Tech Stack:

- Python
- LangChain
- ChromaDB
- OpenAI API
- Streamlit

Project 2: College Notes Assistant

Students upload semester notes and ask:

- definitions,
- summaries,
- exam questions.

Project 3: HR Policy Chatbot

Employees query:

- leave policies,
- payroll rules,
- reimbursement guidelines.

---

2. Advanced RAG

Overview

Advanced RAG improves retrieval quality using:

- hybrid search,
- reranking,
- metadata filtering,
- semantic chunking,
- query rewriting.

Architecture Flow

```
User Query
   ↓
Query Expansion
   ↓
Hybrid Search
   ↓
Reranker
   ↓
Filtered Context
   ↓
LLM
```

Technologies Used

- BM25
- Elasticsearch
- Cohere Rerank
- Cross Encoders
- Weaviate
- Milvus

Advantages

- Better accuracy
- Improved retrieval precision
- Lower hallucinations
- Better enterprise performance

Limitations

- More infrastructure
- Higher cost
- Increased complexity

Example Projects

Project 1: Enterprise Knowledge Assistant

Employees search:

- SOP documents,
- internal guides,
- architecture documents,
- troubleshooting manuals.

Features:

- metadata filtering,
- department-based retrieval,
- reranking.

Project 2: Legal Document Analyzer

Lawyers query:

- contracts,
- legal clauses,
- compliance documents.

System highlights:

- important clauses,
- risks,
- policy violations.

Project 3: Medical Research Assistant

Doctors search:

- journals,
- medical papers,
- patient treatment references.

---

3. Agentic RAG

Overview

Agentic RAG introduces AI agents that can:

- decide retrieval strategies,
- call APIs,
- search multiple databases,
- reason step-by-step,
- perform autonomous workflows.

Architecture Flow

```
User Query
   ↓
AI Agent
 ├── Vector Search
 ├── Web Search
 ├── SQL Query
 ├── API Calls
 ├── Tool Execution
 └── Multi-step Reasoning
   ↓
LLM Response
```

Technologies Used

- LangGraph

- CrewAI
- AutoGen
- OpenAI Function Calling
- MCP Servers
- FastAPI

Advantages

- Dynamic reasoning
- Multi-hop retrieval
- Autonomous workflows
- Tool usage support

Limitations

- High latency
- Expensive inference
- Complex orchestration

Example Projects

Project 1: AI Software Engineering Copilot

Capabilities:

- analyze repositories,
- generate code,
- debug errors,
- create documentation,
- run tests.

Project 2: Intelligent QA Automation Generator

System can:

- read FRD documents,
- generate user stories,
- generate test cases,
- generate workflows,
- create automation scripts.

Project 3: AI Research Assistant

The agent:

- searches web sources,
- summarizes papers,

- compares findings,
- creates reports.

---

4. Graph RAG

Overview

Graph RAG uses knowledge graphs instead of only vector similarity.

Relationships between entities are stored explicitly.

Example Relationship

```
Customer → purchased → Product
Product → belongs to → Category
Category → managed by → Department
```

Architecture Flow

```
User Query
   ↓
Knowledge Graph Traversal
   ↓
Context Retrieval
   ↓
LLM
```

Technologies Used

- Neo4j
- RDF Graphs
- Graph Databases
- Cypher Query Language
- Graph Neural Networks

Advantages

- Better relational reasoning
- Explainable retrieval
- Strong multi-hop intelligence

Limitations

- Difficult setup
- Requires graph engineering
- Expensive maintenance

Example Projects

Project 1: Fraud Detection System

Tracks:

- transactions,
- devices,
- accounts,
- suspicious relationships.

Project 2: Healthcare Knowledge Graph Assistant

Connects:

- diseases,
- symptoms,
- medications,
- treatment plans.

Project 3: Supply Chain Intelligence Platform

Tracks:

- vendors,
- shipments,
- warehouses,
- delays,
- dependencies.

---

5. Hybrid RAG

Overview

Hybrid RAG combines multiple retrieval systems together.

Retrieval Methods Combined

- Vector search
- Keyword search
- Metadata filtering
- Graph retrieval
- SQL retrieval

Architecture Flow

```
User Query
   ↓
Hybrid Retriever
 ├── Vector Search
 ├── BM25 Search
 ├── Graph Retrieval
 └── SQL Queries
   ↓
Reranking
   ↓
LLM
```

Advantages

- High retrieval accuracy
- Better recall
- Production-ready performance

Limitations

- Complex pipelines
- Higher infrastructure cost

Example Projects

Project 1: Enterprise AI Copilot

Supports:

- documents,
- databases,
- dashboards,
- APIs,
- workflows.

Project 2: Banking Intelligence Assistant

Answers:

- transaction questions,
- customer history,
- policy information,
- fraud insights.

Project 3: Manufacturing Operations Assistant

Combines:

- sensor data,
- SOPs,
- machine logs,
- maintenance history.

---

6. Multimodal RAG

Overview

Multimodal RAG retrieves and processes:

- text,
- images,
- audio,
- video,
- tables,
- diagrams.

Architecture Flow

```
Multimodal Input
    ↓
Embedding Models
    ↓
Retriever
    ↓
Vision/Audio LLM
    ↓
Generated Response
```

Technologies Used

- GPT-4o
- Gemini
- Claude
- CLIP
- Whisper
- OCR Pipelines

Advantages

- Handles rich media
- Better contextual understanding
- Supports real-world enterprise workflows

Limitations

- Expensive infrastructure
- Complex embeddings
- Large storage requirements

Example Projects

Project 1: Medical Imaging Assistant

Analyzes:

- X-rays,
- MRI scans,
- patient reports.

Project 2: Video Search Engine

Users search:

- lecture videos,
- surveillance footage,
- tutorials.

Project 3: Intelligent Document Processing System

Processes:

- invoices,
- tables,
- forms,
- scanned PDFs.

7. Corrective RAG (CRAG)

Overview

Corrective RAG validates retrieval quality before generation.

If retrieval quality is poor:

- the system re-retrieves,
- reformulates the query,
- searches alternative sources.

Architecture Flow

```
Retrieve
   ↓
Evaluate Retrieval
   ↓
Correct if Needed
   ↓
Generate Response
```

Advantages

- Reduced hallucinations
- Better retrieval confidence
- Improved reliability

Example Projects

Project 1: Financial Compliance Assistant

Validates retrieved:

- regulations,
- financial policies,
- audit requirements.

Project 2: Pharmaceutical Knowledge Assistant

Ensures:

- medically accurate retrieval,

- validated scientific responses.

---

8. Self-RAG

Overview

Self-RAG allows the model to:

- critique itself,
- evaluate generated answers,
- request additional retrieval if needed.

Architecture Flow

```
Generate
   ↓
Self Evaluation
   ↓
Retrieve More Context
   ↓
Regenerate
```

Advantages

- Improved reasoning
- Better factual accuracy
- Adaptive intelligence

Limitations

- Very high compute cost
- Increased latency

Example Projects

Project 1: Academic Research Assistant

Validates:

- citations,
- paper summaries,
- technical explanations.

Project 2: AI Coding Assistant

Self-checks:

- generated code,
- syntax,
- logic errors.

---

9. Adaptive RAG

Overview

Adaptive RAG dynamically decides:

- whether retrieval is needed,
- which retrieval method to use,
- how much context to retrieve.

Advantages

- Cost optimization
- Faster responses
- Better scalability

Example Projects

Project 1: Smart Customer Support Assistant

Simple queries:

- answered directly.

Complex queries:

- trigger retrieval pipelines.

Project 2: AI Personal Assistant

Chooses:

- web search,
- vector DB,
- memory retrieval,
- API calls.

---

10. Modular RAG

Overview

Modular RAG separates the system into reusable modules.

Modules

- Retriever
- Router
- Memory
- Evaluator
- Reranker
- Generator
- Tool Executor

Advantages

- Easy scaling
- Reusable architecture
- Better maintainability

Example Projects

Project 1: Enterprise AI Platform

Supports multiple business departments using reusable RAG modules.

Project 2: Multi-Agent AI Ecosystem

Different agents share:

- retrievers,
- memory,
- evaluation pipelines.

---

11. Hierarchical RAG

Overview

Hierarchical RAG retrieves information in layers.

Retrieval Hierarchy

```
Document
  ↓
Section
  ↓
Paragraph
  ↓
Sentence
```

Advantages

- Better long-document retrieval
- Reduced irrelevant context
- Efficient context handling

Example Projects

Project 1: Large Policy Document Assistant

Handles:

- government policies,
- enterprise compliance documents,
- legal manuals.

Project 2: Book-Based AI Tutor

Retrieves:

- chapters,
- concepts,
- examples,
- summaries.

---

12. Fusion RAG / Multi-Query RAG

Overview

Fusion RAG creates multiple query reformulations.

Example:

User Query: "How does transformer attention work?"

Generated Queries:

- transformer attention
- self-attention mechanism
- query key value explanation
- attention matrix

Advantages

- Better recall
- Improved search coverage
- Better retrieval diversity

Example Projects

Project 1: Technical Documentation Assistant

Improves retrieval from:

- API docs,
- engineering manuals,
- SDK documentation.

Project 2: AI Coding Search Engine

Finds:

- code snippets,
- implementation examples,
- debugging solutions.

---

13. Long-Context RAG

Overview

Long-context RAG uses models with very large context windows.

Instead of aggressive chunk retrieval, entire documents can be injected.

Technologies Used

- Claude

- Gemini
- GPT-4.1
- Long-context transformers

Advantages

- Better continuity
- Full-document understanding
- Reduced chunk fragmentation

Limitations

- Expensive tokens
- Context dilution
- Slower inference

Example Projects

Project 1: Contract Analysis System

Processes entire:

- agreements,
- legal documents,
- procurement contracts.

Project 2: Meeting Intelligence Platform

Analyzes:

- long meeting transcripts,
- discussions,
- decisions,
- action items.

---

14. SQL RAG / Structured RAG

Overview

SQL RAG retrieves information from structured databases.

Architecture Flow

```
Natural Language Query
   ↓
Text-to-SQL
   ↓
Database Query
   ↓
LLM Explanation
```

Technologies Used

- PostgreSQL
- MySQL
- Snowflake
- DuckDB
- SQL Agents

Advantages

- Accurate structured retrieval
- Strong analytics support
- Real-time querying

Example Projects

Project 1: Business Intelligence Assistant

Queries:

- sales data,
- revenue reports,
- KPIs,
- dashboards.

Project 2: Inventory Management Assistant

Tracks:

- stock,
- warehouses,
- shipments,
- procurement.

---

15. Real-Time RAG

Overview

Real-Time RAG retrieves continuously updated information.

Data Sources

- APIs
- logs
- live databases
- IoT sensors
- streaming systems

Advantages

- Live intelligence
- Dynamic updates
- Real-time monitoring

Limitations

- Infrastructure-heavy
- Requires streaming pipelines

Example Projects

Project 1: Stock Market AI Assistant

Provides:

- live stock analysis,
- trading insights,
- market summaries.

Project 2: Cybersecurity Monitoring System

Analyzes:

- logs,
- threats,
- suspicious activity,
- anomaly detection.

Comparison Table

RAG Type	Complexity	Accuracy	Cost	Best Use Case
Naive RAG	Low	Medium	Low	Simple chatbots
Advanced RAG	Medium	High	Medium	Enterprise search
Agentic RAG	High	Very High	High	AI agents
Graph RAG	High	Very High	High	Relationship intelligence
Hybrid RAG	High	Very High	Medium-High	Production AI systems
Multimodal RAG	High	High	High	Vision and media AI
Self-RAG	Very High	Very High	Very High	Research systems
SQL RAG	Medium	High	Medium	Analytics and BI
Real-Time RAG	High	High	High	Live systems

Best RAG Architectures for Different Domains

Domain	Recommended RAG Type
Enterprise Search	Advanced + Hybrid RAG
AI Copilots	Agentic + Hybrid RAG
Healthcare	Graph + Multimodal RAG
Finance	SQL + Corrective RAG
Cybersecurity	Real-Time + Agentic RAG
Education	Hierarchical + Self-RAG
Research Systems	Self-RAG + Fusion RAG
Manufacturing	Hybrid + Real-Time RAG

Current Industry Trends

Modern enterprise AI systems increasingly combine multiple RAG architectures together.

Example:

```
Agentic + Hybrid + Multimodal + Graph RAG
```

This enables:

- autonomous reasoning,
- multi-source retrieval,
- media understanding,
- structured intelligence,
- workflow automation.

The most important RAG architectures currently dominating the industry are:

1. Agentic RAG
2. Hybrid RAG
3. Multimodal RAG
4. Graph RAG
5. Self-Correcting RAG Systems

Conclusion

RAG systems are evolving rapidly from simple vector search pipelines into intelligent autonomous AI ecosystems.

Different RAG architectures solve different problems:

- Naive RAG for simple assistants,
- Advanced RAG for enterprise retrieval,
- Agentic RAG for autonomous workflows,
- Graph RAG for relational intelligence,
- Multimodal RAG for media understanding,
- Real-Time RAG for live systems.

Choosing the right architecture depends on:

- project complexity,
- latency requirements,
- budget,
- accuracy expectations,
- infrastructure scale,
- business objectives.

The future of enterprise AI will heavily depend on combining multiple RAG architectures into unified intelligent systems.